

2026-09-14 更新：两个并行推进的实验 (decomposer 训练数据消融 / 检索 precision 对比)

这一阶段同时做了两个实验，分别看 pipeline 的两个不同环节，互相独立，分两个部分记录：

	看的是哪个环节	变量	对照
第一部分	问题拆解——decomposer 的训练数据	去掉 2Wiki 自标注数据，只用 MuSiQue 训练 decomposer	线上 decomposer (MuSiQue + 2Wiki)，即 0831 报告的 D 组
第二部分	检索——检索结果里有多少是噪声	在 recall 之外补充 precision 口径	D 组 vs ChainRAG vs GRITHopper (0831 报告的三方对比)

两部分的端到端配置都是 D 组 (gate v3, $\lambda=0.60$, distractor setting, comparison_hint reader)。第一部分只换了 拆解文件，第二部分完全没改 D 组配置，只是补了一个新的评测口径。

第一部分：decomposer 训练数据消融——2Wiki 自标注数据贡献了多少

一、问题

端到端评测里的 BART decomposer 是用两份数据混合训练的：MuSiQue 官方的问题分解标注 (19,938 条)，加上 用 GPT 给 2Wiki 训练集自标注的分解 (10,444 条)。MuSiQue 自带分解标注，2Wiki 没有，所以后者是我们自己 额外构造的。这个实验要回答：**如果只用 MuSiQue 自带的标注训练 decomposer，端到端效果会掉多少**，也就是 2Wiki 自标注数据到底贡献了多少。

2Wiki 标注数据是从 2Wiki 训练集里**按题型定额采样**的：共选 10,500 条 (bridge_comparison 5000 / comparison 2500 / compositional 1500 / inference 1500)，用 gpt-5-mini 按 MuSiQue 的分解格式标注，成功 10,444 条 (56 条失败丢弃)，全部进入训练集，在 30,382 条混合训练数据里占 34%。另有 1,043 条 2Wiki dev 标注，只跟 MuSiQue dev 一起用于挑 checkpoint，不参与训练。

题型	原始 2Wiki 训练集	占比	标注后进入训练集	占比	采样率
bridge_comparison (4 跳)	34,631	20.7%	4,998	47.9%	14.4%
comparison	51,963	31.0%	2,453	23.5%	4.7%
compositional	76,481	45.7%	1,500	14.4%	2.0%
inference	4,379	2.6%	1,493	14.3%	34.1%
合计	167,454		10,444		6.2%

按跳数分：2 跳 5,446 条，4 跳 4,998 条（4 跳全部是 bridge_comparison）。采样刻意偏向比较类题：比较类（bridge_comparison + comparison）在原始训练集里占 52%，在标注数据里占 71%，其中需要 4 跳的 bridge_comparison 从 20.7% 提到了 47.9%。compositional 这类跟 MuSiQue 结构相近的桥接题只采了 2%。

二、实验设置

- 两个 decomposer 唯一的区别是训练数据：同样是 BART-large，同样的超参（lr 3e-5、batch 16、3 epoch、seed 100）。只用 MuSiQue 的版本训练集就是那 19,938 条，混合版本是 19,938 + 10,444 条。两者最终都用 第 3 个 epoch 的 checkpoint。
- 下游完全不变：两份拆解都经过同一套流程（BART 输出 → 转成带 [Answer N] 占位符的格式 → Llama 改写成 自然语言子问题），再送进同一个 D 组 pipeline，检索、gate、reader、prompt、λ 全部相同。
- 对照组就是 0831 报告第 1.5 节 D 组那一行，没有重跑。为了知道多大的差距算真实差异，同一配置又完整 重跑了一遍 D 组：MuSiQue 上各指标波动不超过 0.2pt，2Wiki 不超过 0.06pt，HotpotQA 逐位一致。下面的差距 都远大于这个噪声，只有 MuSiQue 上零点几个点的差异需要谨慎看。
- 评测口径跟 D 组一致：recall@1 / recall@3 / chain@1 都是按位置对齐的（第 j 跳检索到的段落要跟标注里 第 j 跳的 gold 对上），EM / F1 是 HotpotQA 官方口径。

三、decomposer 本身的表现

MuSiQue dev（有官方分解标注，能直接算）：

decomposer	BLEU	跳数准确率	2 跳题	3 跳题	4 跳题
MuSiQue + 2Wiki（线上）	76.81	0.8366	0.982	0.782	0.489
只用 MuSiQue	75.94	0.8155	0.988	0.738	0.427

2Wiki / HotpotQA dev（没有分解标注，只能拿拆出来的跳数跟 supporting facts 数量比）：

数据集	decomposer	跳数准确率	拆出来的跳数分布	gold 跳数分布
2Wiki	MuSiQue + 2Wiki	0.9651	2 跳 9689 / 3 跳 140 / 4 跳 2739 / 5 跳 8	2 跳 9595 / 3 跳 88 / 4 跳 2806 / 5+ 跳 87
	只用 MuSiQue	0.7616	1 跳 10 / 2 跳 12428 / 3 跳 138 / 4 跳 0	同上
HotpotQA	MuSiQue + 2Wiki	0.8763	1 跳 14 / 2 跳 6489 / 3 跳 853 / 4+ 跳 49	全部 2 跳
	只用 MuSiQue	0.9369	1 跳 40 / 2 跳 6938 / 3 跳 422 / 4+ 跳 5	同上

两个现象：

1. 只用 **MuSiQue** 训练的模型完全不会拆出 **4 跳**——2Wiki 上 1.26 万道题一道都没有，而 2Wiki 有 2806 道 4 跳题。在 MuSiQue 上它也倾向于少拆，4 跳题有一半多被拆成 3 跳。

2. HotpotQA 全部是 2 跳题，只用 MuSiQue 的模型拆得更"保守"，跳数准确率反而更高（0.937 vs 0.876）。

四、端到端结果

每格是只用 MuSiQue 训练的结果，括号里是相对线上 decomposer（D 组）的变化，单位 pt：

数据集	recall@1	recall@3	chain@1	EM	F1
MuSiQue	0.7069 (+0.8)	0.8474 (+0.4)	0.5275 (+1.6)	0.4419 (+0.3)	0.5363 (+0.2)
2WikiMultihopQA	0.6907 (-24.1)	0.8689 (-11.5)	0.5651 (-30.3)	0.5189 (-8.3)	0.5976 (-8.4)
HotpotQA	0.5825 (-8.0)	0.8290 (-4.1)	0.4704 (-9.1)	0.5423 (-1.0)	0.6792 (-0.9)

对照组 D 组的原始数字：MuSiQue 0.6994 / 0.8435 / 0.5118 / 0.4386 / 0.5341，2Wiki 0.9316 / 0.9842 / 0.8685 / 0.6023 / 0.6819，HotpotQA 0.6620 / 0.8701 / 0.5610 / 0.5519 / 0.6883。

五、分析：掉分集中在哪里

5.1 2Wiki：掉分几乎全部来自比较类题

按 2Wiki 官方题型拆开：

题型	题数	EM 变化	F1 变化	只用 MuSiQue 的模型拆出的跳数
bridge_comparison（4 跳）	2751	-20.1pt	-19.7pt	2 跳 2621 / 3 跳 130
comparison	3040	-12.6pt	-12.2pt	基本都是 2 跳
inference	1549	-4.8pt	-4.8pt	2 跳
compositional	5236	-0.8pt	-1.4pt	2 跳

- **compositional（桥接题）几乎不受影响**：这类题"先找 A，再用 A 找 B"的结构跟 MuSiQue 一样，只用 MuSiQue 训练已经学会了。
- **比较类题掉得最多**："A 和 B 谁更早出生"这种题，正确拆法是先分别查 A、B，再比较。MuSiQue 里几乎没有这种 结构，所以只用 MuSiQue 的模型不会拆。bridge_comparison（"A 的导演和 B 的导演谁更年长"）需要 4 跳，而这个 模型从来不拆 4 跳，全部压成 2 跳，EM 掉了 20pt。

结论：2Wiki 自标注数据主要教会 decomposer 怎么拆比较类问题，包括需要 4 跳的复合比较。这类题在 2Wiki 里占 46%（5791 / 12576），所以去掉这份数据，2Wiki 整体掉 8pt。这也跟标注数据的构成对得上（见一节表格）：标注数据里比较类题占了 71%，正是去掉之后掉分最多的两类；compositional 在标注数据里只占 14%，而且结构跟 MuSiQue 相近，去掉以后几乎没有影响。

5.2 MuSiQue：总体略升，但 4 跳题在掉

按标注跳数拆开：

gold 跳数	题数	EM 变化	F1 变化
2 跳	1252	+1.8pt	+2.2pt
3 跳	760	−0.1pt	−1.3pt
4 跳	405	−3.5pt	−3.1pt

总体小幅上升是 2 跳题（占一半多）拉上去的，越长的链掉得越多，跟三节里"只用 MuSiQue 的模型倾向于少拆" 一致。加入 **2Wiki 数据对 MuSiQue 本身的长链题也有帮助**，这跟 decomposer 自身指标（4 跳题跳数准确率 0.427 → 0.489）对得上：2Wiki 里 4 条证据的复合比较题让模型学会了拆长链。

另外，只用 MuSiQue 的模型拆的跳数更少，按位置对齐计分的只有 5964 跳（对照组 6037 跳），更难的后几跳被计分 得少了，所以检索指标的小幅上升有一部分来自这里，不全是检索真的变好。

5.3 HotpotQA：同样是比较类题在掉

HotpotQA 全是 2 跳题，只用 MuSiQue 的模型跳数拆得更准，但按位置对齐的检索指标掉了 8-9pt，EM/F1 只掉约 1pt。按 HotpotQA 官方题型拆开：

题型	题数	EM 变化	F1 变化	每跳选中 gold（按位置对齐）	每跳选中 gold（任意一篇 gold 都算）
bridge	5918	−0.2pt	+0.0pt	0.652 → 0.627	0.895 → 0.893
comparison	1487	−4.1pt	−4.6pt	0.699 → 0.396	0.896 → 0.821

- 答案层面，损失全部来自比较类题，跟 2Wiki 的结论一致；bridge 题完全不受影响。
- 检索层面，按位置对齐的掉分大部分是顺序问题：比较类题"A 和 B 哪个....."，拆成先查 A 还是先查 B 都对，但标注里只有一种顺序，按位置对齐会把顺序反过来也算成错。只看"选中的是不是这道题的任意一篇 gold"，bridge 题几乎没变（−0.2pt），比较类题掉了 7.5pt——这部分才是真实的检索损失（拆错导致查错段落），其余 23pt 左右是顺序。

六、小结

- **2Wiki 自标注数据是必要的**：去掉以后 2Wiki 端到端 EM −8.3pt、chain@1 −30pt，损失几乎全部来自比较类题（bridge_comparison EM −20pt、comparison −12.6pt）。
- **对 MuSiQue 本身的影响是结构性的**：总体持平，但 4 跳题 EM −3.5pt，被 2 跳题的小幅提升抵消了。
- **对 HotpotQA 影响小，但机制相同**：整体答案只掉约 1pt，全部来自比较类题（comparison EM −4.1pt，bridge −0.2pt）；HotpotQA 里比较类题只占 20%，所以整体影响远小于 2Wiki。
- **三个数据集指向同一个结论**：2Wiki 自标注数据的价值在于教会 decomposer 拆**比较类问题**和**长链问题**，这两类结构在 MuSiQue 的官方标注里几乎没有或很少。

第二部分：检索 precision 对比——D 组 vs ChainRAG vs GRITHopper

一、动机

0831 报告的三方对比只看了 recall 类指标（逐跳 recall@1/@3、每题 gold 覆盖率），没有回答"检索到的东西里 有多少是噪声"。这一部分补上 precision。数据全部来自已有的三方逐题输出；只有 D 组为了保留每跳的候选排序， 用完全相同的配置重跑了一遍（重跑结果跟报告数字的差距在上面第一部分二节说的噪声范围内）。

跟 0831 一样，三方对比只能用顺序无关口径（GRITHopper、ChainRAG 的跳跟 gold 之间没有位置对应关系）， 并且只能按段落标题匹配（这两个方法的输出只存了标题）。

二、两种 precision

1. 证据集合 precision：每道题最终用来生成答案的证据集合里，有多少比例是 gold。

precision = $\frac{|\text{检索集合} \cap \text{gold}|}{|\text{检索集合}|}$

"检索集合"跟 0831 的 gold 覆盖率用的是同一个集合（D 组、GRITHopper 是每跳最终选定的那一篇；ChainRAG 是 图扩展后塞进 context 的所有句子对应的段落）。覆盖率就是把分母换成 gold 数。

对 D 组和 GRITHopper，这个指标数值上几乎就等于 recall@1：两者每跳只拿 1 篇，拿的就是排序第一，"最终证据里有多少是 gold"和"每跳 top-1 是不是 gold"数的是同一批东西（差距不到 0.5pt）。所以它真正新增的 信息只在 ChainRAG 上。

2. 逐跳 precision@3：每一跳排序前 3 名里有几篇 gold，再除以 3。

它跟 recall@3 的区别：0831 报告里的"recall@3"（严格说是 hit@3）只看前 3 名里有没有至少一篇 gold， 结果是 0 或 1；precision@3 看有几篇。前 3 名是 [gold A, 噪声, 噪声] 和 [gold A, gold B, 噪声] 时， recall@3 都是 1， precision@3 分别是 1/3 和 2/3。三个方法的 recall@3 都挤在 0.96-0.99，分不开， precision@3 能分开。

注意 precision@3 有上限：一题只有 2 篇 gold 时最高 2/3。HotpotQA 每题都是 2 篇，上限就是 0.667， 所以 不同数据集之间不能比 precision@3 的绝对值，只能在同一数据集内比较方法。

三、结果

3.1 证据集合 precision

数据集	方法	precision	覆盖率	集合 F1	平均检索篇数	平均 gold 篇数
MuSiQue	D 组	0.8837	0.7958	0.8275	2.28	2.60
	ChainRAG	0.2068	0.9587	0.3235	14.54	
	GRITHopper	0.9091	0.8207	0.8517	2.31	
2WikiMultihopQA	D 组	0.9673	0.9408	0.9496	2.37	2.44
	ChainRAG	0.3559	0.9870	0.4998	7.61	
	GRITHopper	0.9540	0.8922	0.9139	2.24	
HotpotQA	D 组	0.8955	0.8532	0.8650	1.92	2.00
	ChainRAG	0.2752	0.9864	0.4215	7.86	

数据集	方法	precision	覆盖率	集合 F1	平均检索篇数	平均 gold 篇数
	GRITHopper	0.9658	0.9464	0.9528	1.95	

(precision、覆盖率、集合 F1 都是每题算完再平均。)

3.2 逐跳 precision@3

数据集	方法	跳数	recall@1	recall@3	precision@3	前 3 名平均 gold 篇数
MuSiQue	D 组	6037	0.8836	0.9742	0.5126	1.54
	ChainRAG	5162	0.6716	0.8628	0.3949	1.18
	GRITHopper	6404	0.9058	0.9863	0.6087	1.83
2WikiMultihopQA	D 组	30621	0.9724	0.9972	0.6158	1.85
	ChainRAG	22737	0.9199	0.9785	0.5859	1.76
	GRITHopper	30654	0.9500	0.9964	0.6786	2.04
HotpotQA	D 组	14796	0.8988	0.9805	0.5035	1.51
	ChainRAG	13658	0.8032	0.9496	0.4679	1.40
	GRITHopper	14810	0.9658	0.9959	0.5994	1.80

GRITHopper、ChainRAG 的 recall@1/@3 跟 0831 报告逐位一致。D 组 MuSiQue 的 recall@1 这里是 0.8836，报告里是 0.8552，差别来自匹配方式：报告按段落编号匹配，这里只能按标题匹配，MuSiQue 里有同名段落（约 6.7% 的题 D 组 选中了一篇跟 gold 同名但不是 gold 的段落，按标题会被记成命中）。2Wiki、HotpotQA 两种匹配方式结果一样。GRITHopper、ChainRAG 在 MuSiQue 上大概也有同样程度的虚高，只是没法量化。

四、解读

ChainRAG 的高覆盖率是用数量换来的。 MuSiQue 上它平均每题送给 reader 14.5 篇，gold 只有 2.6 篇，context 里约 80% 是噪声。这量化了 0831 报告 2.5.1 节的解释（图扩展"广撒网"稀释了正确信号）：覆盖率全场 最高，EM/F1 却全场最低。把 precision 和覆盖率合成集合 F1 后，排名变成 GRITHopper ≈ D 组 >> ChainRAG。

逐跳 precision@3 三个数据集一致：GRITHopper > D 组 > ChainRAG。 但 D 组和 GRITHopper 之间的差距要结合 查询方式来读：

- GRITHopper 每一步都用**完整原问题**做查询，所以这道题所有的 gold 段落在每一步都会排得靠前——MuSiQue 上 前 3 名平均有 1.83 篇 gold。
- D 组先拆成子问题，每一跳的查询只针对**这一跳要找的那一篇**，别的跳的 gold 排不上来，本来就是查询足够 精准的表现——前 3 名平均 1.54 篇 gold。

所以 precision@3 在这里部分衡量的是"查询有多聚焦", 拆解类方法 (D 组、ChainRAG) 天然吃亏。**D 组比 GRITHopper 低**, 主要反映的是两种查询方式的差异, 不代表 **D 组检索更差**——D 组每跳 top-1 的命中率在 2Wiki 上 还高于 GRITHopper (0.9724 vs 0.9500), 而这才是 D 组实际提交的证据。

D 组和 ChainRAG 都是拆解类方法, 查询方式可比。 D 组的 precision@3 在三个数据集上都高于 ChainRAG (MuSiQue +11.8pt、2Wiki +3.0pt、HotpotQA +3.6pt), recall@1 也全面领先: 前 3 名里 gold 更多、噪声更少。不过两者的拆解 (BART decomposer vs ChainRAG 自己用 LLM 拆) 和排序 (BGE + gate 重排 vs embedding + cross-encoder) 都不一样, 这个差距不能单独归因到 gate 上。